

KUBERNETES (K8S) TASK - 02

1. Create a Simple Pod Using YAML Task: Write a YAML file to create a Pod named firstpod with an nginx container.

Verify the Pod creation using `kubectl get pods` and check the logs of the container using `kubectl logs firstpod`.

Objective: Create a Kubernetes Pod named **firstpod** using a YAML file. The Pod will contain a **NGINX container**. Then verify the Pod and check its logs using `kubectl`.

1. Prerequisites

Before starting, make sure:

- Kubernetes cluster is running.
- You have **1 Master/Control Plane + Worker Node(s)**.
- `kubectl` is installed.
- `kubectl` is configured to communicate with the cluster.

Check the cluster:

```
kubectl get nodes
```

Expected output will look similar to:

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	Ready	control-plane	20m	v1.34.x
k8s-worker1	Ready	<none>	15m	v1.34.x
k8s-worker2	Ready	<none>	15m	v1.34.x

All nodes should show:

```
STATUS Ready
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get nodes  
NAME             STATUS    ROLES    AGE   VERSION  
k8s-master       Ready    control-plane   18h   v1.34.11  
k8s-worker1      Ready    <none>         18h   v1.34.11  
k8s-worker2      Ready    <none>         18h   v1.34.11  
ubuntu@k8s-master:~$ |
```

2. Create a YAML File

Create a file named:

firstpod.yaml

You can use vi:

Vi firstpod.yaml

3. Write the Pod YAML

Add the following content:

apiVersion: v1

kind: Pod

metadata:

name: firstpod

spec:

containers:

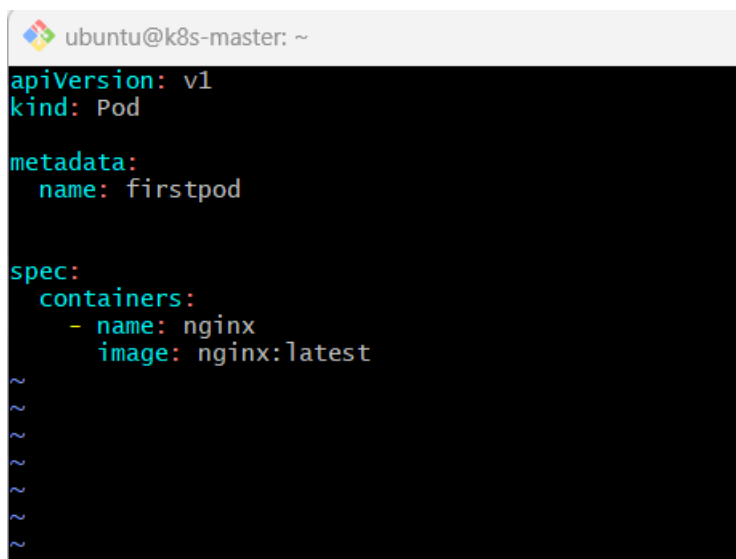
- name: nginx

image: nginx:latest

Explanation

Field	Meaning
apiVersion: v1	Kubernetes API version
kind: Pod	Tells Kubernetes to create a Pod
metadata	Contains Pod information
name: firstpod	Name of the Pod
spec	Defines the desired Pod configuration
containers	List of containers inside the Pod
name: nginx	Container name
image: nginx:latest	NGINX Docker image

4. Save the File

A terminal window with a black background and a light gray title bar. The title bar contains the Ubuntu logo and the text 'ubuntu@k8s-master: ~'. The terminal shows the following text: 'apiVersion: v1', 'kind: Pod', 'metadata:', ' name: firstpod', 'spec:', ' containers:', ' - name: nginx', ' image: nginx:latest'. There are several tilde '~' characters on the left side of the terminal, indicating a scrollable history. The text is color-coded: 'apiVersion' is red, 'kind' is green, 'metadata' is blue, 'spec' is red, 'containers' is green, and the list item is blue.

```
ubuntu@k8s-master: ~  
apiVersion: v1  
kind: Pod  
metadata:  
  name: firstpod  
spec:  
  containers:  
    - name: nginx  
      image: nginx:latest  
~  
~  
~  
~  
~  
~  
~
```

5. Verify the YAML File

Run:

```
cat firstpod.yaml
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ vi firstpod.yaml  
ubuntu@k8s-master:~$ cat firstpod.yaml  
apiVersion: v1  
kind: pod  
  
metadata:  
  name: firstpod  
  
spec:  
  containers:  
  - name: nginx  
    image: nginx:latest  
ubuntu@k8s-master:~$ |
```

6. Create the Pod

Use:

```
kubectl apply -f firstpod.yaml
```

Expected output:

```
pod/firstpod created
```

This means Kubernetes has accepted the YAML and created the Pod.

7. Verify Pod Creation

Run:

```
kubectl get pods
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
firstpod	1/1	Running	0	10s

The important information is:

```
firstpod 1/1 Running
```

What does 1/1 mean?

It means:

- **1 container is expected**

- 1 container is currently running/ready

8. Get More Information About the Pod

Run:

`kubectl get pod firstpod -o wide`

```

ubuntu@k8s-master: ~$ kubectl apply -f firstpod.yaml
pod/firstpod created
ubuntu@k8s-master:~$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
firstpod   1/1     Running   0           23s
nginx      1/1     Running   1 (66m ago) 19h
ubuntu@k8s-master:~$ kubectl get pod firstpod -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
firstpod   1/1     Running   0           66s   192.168.126.1 k8s-worker2 <none>
e>
ubuntu@k8s-master:~$ |

```

9. Describe the Pod

Run:

`kubectl describe pod firstpod`

This gives detailed information about:

- Pod name
- Namespace
- Node
- Container
- Image
- IP address
- Events
- Container state

```

ubuntu@k8s-master: ~$ kubectl describe pod firstpod
Name: firstpod
Namespace: default
Priority: 0
Service Account: default
Node: k8s-worker2/172.31.26.51
Start Time: Wed, 23 Sep 2026 05:55:23 +0000
Labels: <none>
Annotations: cni.projectcalico.org/containerID: 754665d3f53abf3a9c683bcc7ff
              aa21e856766fd13d32d95e5eb42a3c7f44ecf
              cni.projectcalico.org/podIP: 192.168.126.1/32
              cni.projectcalico.org/podIPs: 192.168.126.1/32
Status: Running
IP: 192.168.126.1
IPs:
  IP: 192.168.126.1
Containers:
  nginx:
    Container ID: containerd://96ffbd14c552c823b4f61100a16c2a88b8f6571912eb503
    00b2a8ded69ffcf9d
    Image: nginx:latest
    Image ID: docker.io/library/nginx@sha256:abe47724e466aeab9a345d8e46a22
    1c2fa8953c7848bb4a3bd9976a7199f8cf2
    Port: <none>
    Host Port: <none>
    State: Running
      Started: Wed, 23 Sep 2026 05:55:26 +0000
    Ready: True
    Restart Count: 0
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-pdg6j (
ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized        True
  Ready              True
  ContainersReady    True
  PodScheduled       True
Volumes:
  kube-api-access-pdg6j:
    Type: Projected (a volume that contains injected data fro
multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional: false
    DownwardAPI: true
QoS Class: BestEffort
Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists fo
r 300s
              node.kubernetes.io/unreachable:NoExecute op=Exists
for 300s
Events:
  Type    Reason      Age    From          Message
  ----    -
Normal   Scheduled   4m2s   default-scheduler   Successfully assigned default/fir

```

10. Check the Container Logs

Now perform the task requested:

`kubectl logs firstpod`

You may see output similar to:

`/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration`

`/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/`

`/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh`

...

```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl logs firstpod
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/09/23 05:55:26 [notice] 1#1: using the "epoll" event method
2026/09/23 05:55:26 [notice] 1#1: nginx/1.31.6
2026/09/23 05:55:26 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/09/23 05:55:26 [notice] 1#1: OS: Linux 7.0.0-1006-aws
2026/09/23 05:55:26 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/09/23 05:55:26 [notice] 1#1: start worker processes
2026/09/23 05:55:26 [notice] 1#1: start worker process 29
2026/09/23 05:55:26 [notice] 1#1: start worker process 30
ubuntu@k8s-master:~$

```

11. Check the Pod Again

Run:

```
kubectl get pods
```

Expected:

NAME	READY	STATUS	RESTARTS	AGE
firstpod	1/1	Running	0	2m

12. Check the NGINX Container

You can verify the container information using:

```
kubectl get pod firstpod -o jsonpath='{.spec.containers[*].name}'
```

Expected:

```
nginx
```

Check the image:

```
kubectl get pod firstpod -o jsonpath='{.spec.containers[*].image}'
```

Expected:

```
nginx:latest
```

13. Verify NGINX Is Running Inside the Pod

Execute a command inside the container:

```
kubectl exec -it firstpod -- nginx -v
```

Expected output:

```
nginx version: nginx/1.x.x
```

The exact version depends on the current nginx:latest image.

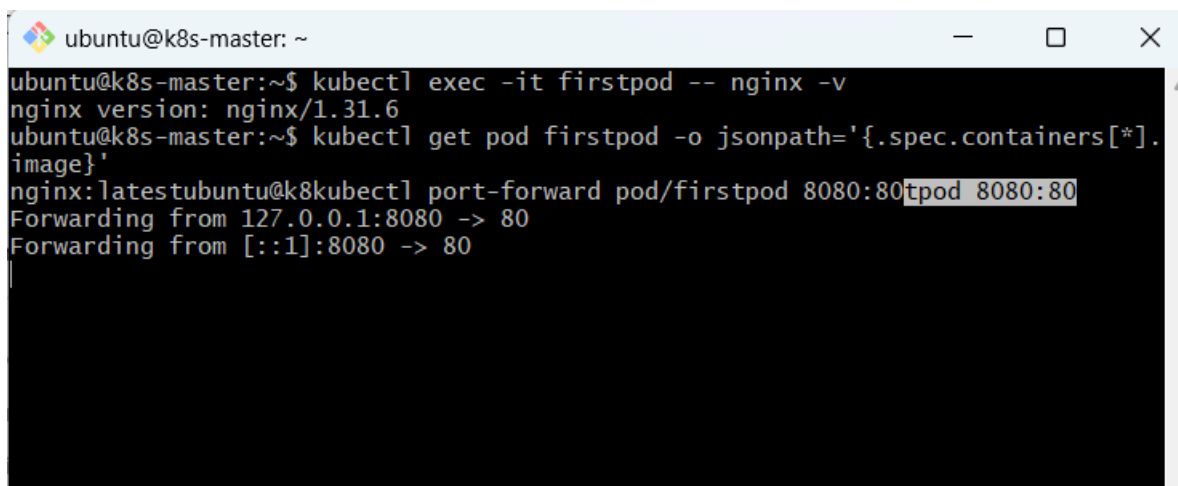
You can also check the NGINX process:

```
kubectl exec firstpod -- ps
```

14. Test NGINX

You can test NGINX from your local machine by creating a temporary port-forward:

```
kubectl port-forward pod/firstpod 8080:80
```



```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl exec -it firstpod -- nginx -v  
nginx version: nginx/1.31.6  
ubuntu@k8s-master:~$ kubectl get pod firstpod -o jsonpath='{.spec.containers[*].  
image}'  
nginx:latest  
ubuntu@k8s-master:~$ kubectl port-forward pod/firstpod 8080:80  
Forwarding from 127.0.0.1:8080 -> 80  
Forwarding from [::1]:8080 -> 80
```

2.Set Environment Variables in a Pod Task: Modify the YAML file to include environment variables myname: raji and City: Hyderabad. Deploy the Pod and use kubectl exec <pod_name> -- env to check if the environment variables are set properly.

Set Environment Variables in a Pod

Objective

Modify the Pod YAML file to set two environment variables inside an NGINX container:

- myname = raji
- City = Hyderabad

Then deploy the Pod and verify the variables using:

```
ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl exec firstpod -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=firstpod
NGINX_VERSION=1.31.6
NJS_VERSION=1.0.1
NJS_RELEASE=1~trixie
ACME_VERSION=0.4.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
HOME=/root
ubuntu@k8s-master:~$ |
```

1. Prerequisites

Make sure your Kubernetes cluster is running:

`kubectl get nodes`

Expected:

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	Ready	control-plane	...	...
k8s-worker1	Ready	<none>	...	...
k8s-worker2	Ready	<none>	...	...

The nodes should be in Ready state.

2. Create/Edit the YAML File

Create a file:

```
ubuntu@k8s-master: ~  
apiVersion: v1  
kind: Pod  
  
metadata:  
  name: firstpod  
  
spec:  
  containers:  
  - name: nginx  
    image: nginx:latest  
    env:  
    - name: myname  
      value: "raji"  
    - name: City  
      value: "Hyd"  
~  
~  
~  
~  
~
```

Important

The env section is under the container:

containers:

- name: nginx
image: nginx:latest
env:

Make sure the indentation is correct.

3. Understand the YAML

Pod name

metadata:

name: firstpod

This creates a Pod named:

firstpod

Container

containers:

- name: nginx
image: nginx:latest

The Pod contains an NGINX container.

Environment variables

env:

- name: myname
value: "sabair"
- name: City
value: "Hyderabad"

This creates:

myname=sabair

City=Hyderabad

inside the NGINX container.

4. Save the File

5. Verify the YAML

Run:

cat firstpod.yaml

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get nodes  
NAME           STATUS    ROLES    AGE   VERSION  
k8s-master     Ready     control-plane  20h   v1.34.11  
k8s-worker1    Ready     <none>      20h   v1.34.11  
k8s-worker2    Ready     <none>      20h   v1.34.11  
ubuntu@k8s-master:~$ vi firstpod.yaml  
ubuntu@k8s-master:~$ cat firstpod.yaml  
apiVersion: v1  
kind: Pod  
  
metadata:  
  name: firstpod  
  
spec:  
  containers:  
  - name: nginx  
    image: nginx:latest  
    env:  
    - name: myname  
      value: "raji"  
    - name: City  
      value: "Hyd"  
ubuntu@k8s-master:~$
```

6. Deploy the Pod

Run:

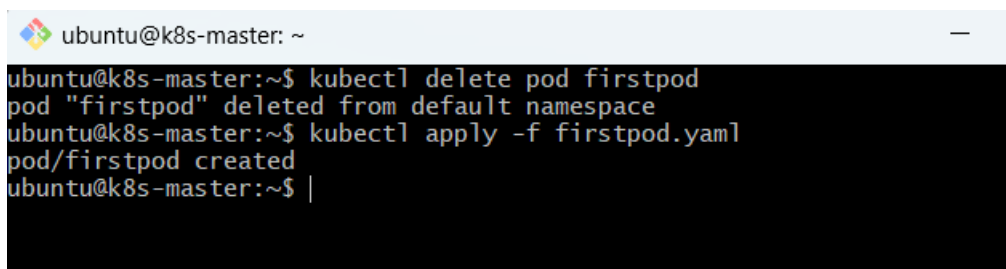
```
kubectl apply -f firstpod.yaml
```

Expected output:

pod/firstpod created

If the Pod already exists, you may see:

pod/firstpod configured

A terminal window with a light blue header bar showing 'ubuntu@k8s-master: ~'. The terminal output is as follows:

```
ubuntu@k8s-master:~$ kubectl delete pod firstpod
pod "firstpod" deleted from default namespace
ubuntu@k8s-master:~$ kubectl apply -f firstpod.yaml
pod/firstpod created
ubuntu@k8s-master:~$ |
```

7. Check the Pod Status

Run:

```
kubectl get pods
```

Expected:

NAME	READY	STATUS	RESTARTS	AGE
firstpod	1/1	Running	0	10s

Wait until the status becomes:

Running

8. Check Environment Variables

Now run the command from the task:

```
kubectl exec firstpod -- env
```

You should see many environment variables. Among them should be:

myname=sabair

City=Hyderabad

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get pod  
NAME      READY   STATUS    RESTARTS   AGE  
firstpod  1/1     Running   0           52s  
ubuntu@k8s-master:~$ kubectl exec firstpod -- env  
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin  
HOSTNAME=firstpod  
NGINX_VERSION=1.31.6  
NJS_VERSION=1.0.1  
NJS_RELEASE=1~trixie  
ACME_VERSION=0.4.1  
PKG_RELEASE=1~trixie  
DYNPKG_RELEASE=1~trixie  
City=Hyderbad  
myname=raji  
KUBERNETES_SERVICE_PORT=443  
KUBERNETES_SERVICE_PORT_HTTPS=443  
KUBERNETES_PORT=tcp://10.96.0.1:443  
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443  
KUBERNETES_PORT_443_TCP_PROTO=tcp  
KUBERNETES_PORT_443_TCP_PORT=443  
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1  
KUBERNETES_SERVICE_HOST=10.96.0.1  
HOME=/root  
ubuntu@k8s-master:~$
```

9. Check Only myname

Instead of displaying all environment variables, you can run:

```
kubectl exec firstpod -- printenv myname
```

Expected:

sabair

10. Check Only City

Run:

```
kubectl exec firstpod -- printenv City
```

Expected:

Hyderabad

11. Verify Both Variables

You can use:

```
kubectl exec firstpod -- sh -c 'echo $myname'
```

Expected:

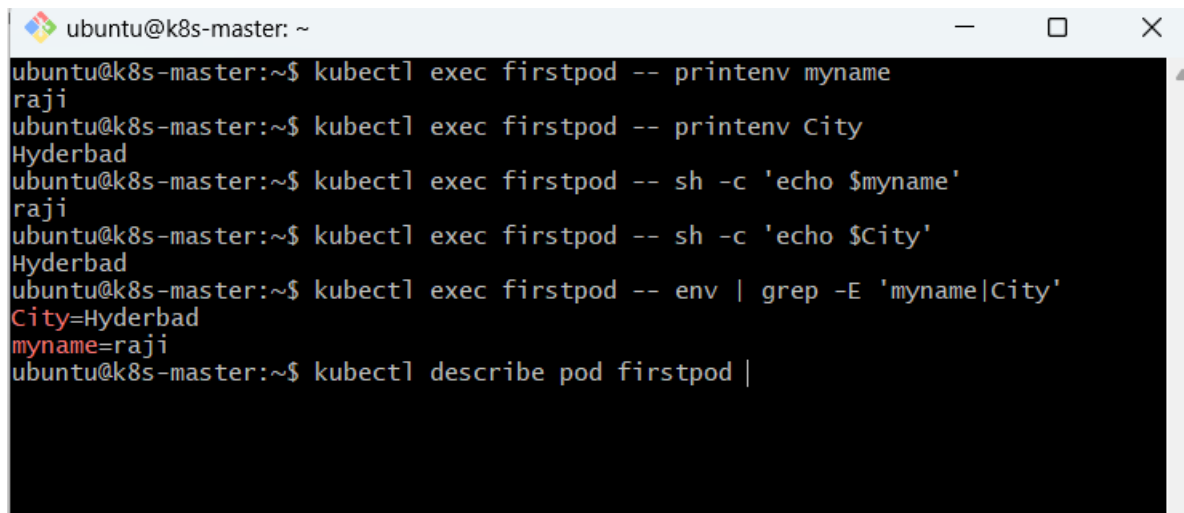
sabair

And:

```
kubectl exec firstpod -- sh -c 'echo $City'
```

Expected:

Hyderabad



```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl exec firstpod -- printenv myname  
raji  
ubuntu@k8s-master:~$ kubectl exec firstpod -- printenv City  
Hyderabad  
ubuntu@k8s-master:~$ kubectl exec firstpod -- sh -c 'echo $myname'  
raji  
ubuntu@k8s-master:~$ kubectl exec firstpod -- sh -c 'echo $City'  
Hyderabad  
ubuntu@k8s-master:~$ kubectl exec firstpod -- env | grep -E 'myname|City'  
City=Hyderabad  
myname=raji  
ubuntu@k8s-master:~$ kubectl describe pod firstpod |
```

13. Verify From Pod Description

You can also run:

```
kubectl describe pod firstpod
```

Look for the container section:

Containers:

nginx:

Image: nginx:latest

Environment:

myname: sabair

City: Hyderabad

This confirms that Kubernetes configured the variables for the container.

```
ubuntu@k8s-master: ~  
Containers:  
nginx:  
  Container ID:   containerd://d713691f3740883900baa97b7e71ef8637573e75ea94cea  
08b8ecf0c7243531d  
  Image:          nginx:latest  
  Image ID:       docker.io/library/nginx@sha256:abe47724e466aeab9a345d8e46a22  
1c2fa8953c7848bb4a3bd9976a7199f8cf2  
  Port:          <none>  
  Host Port:     <none>  
  State:         Running  
    Started:     Wed, 23 Sep 2026 06:36:54 +0000  
  Ready:         True  
  Restart Count: 0  
  Environment:  
    myname:      raji  
    City:        Hyderabad  
  Mounts:  
    /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-jjpcb (ro)  
Conditions:  
  Type                               Status  
PodReadyToStartContainers           True  
Initialized                         True  
Ready                               True
```

3. Deploy a Pod with Commands (Args) in YAML Task: Modify the YAML file to add args that instruct the container to sleep for 50 seconds. Deploy the Pod and use kubectl describe pod to verify the args are correctly passed to the container.

Deploy a Pod with Commands (Args) in YAML

Objective

Create a Kubernetes Pod named **firstpod** and configure the container to **sleep for 50 seconds** using the args field in the YAML file.

Then use:

kubectl describe pod firstpod

to verify that the arguments were passed correctly to the container.

1. Prerequisites

Check that your Kubernetes nodes are ready:

kubectl get nodes

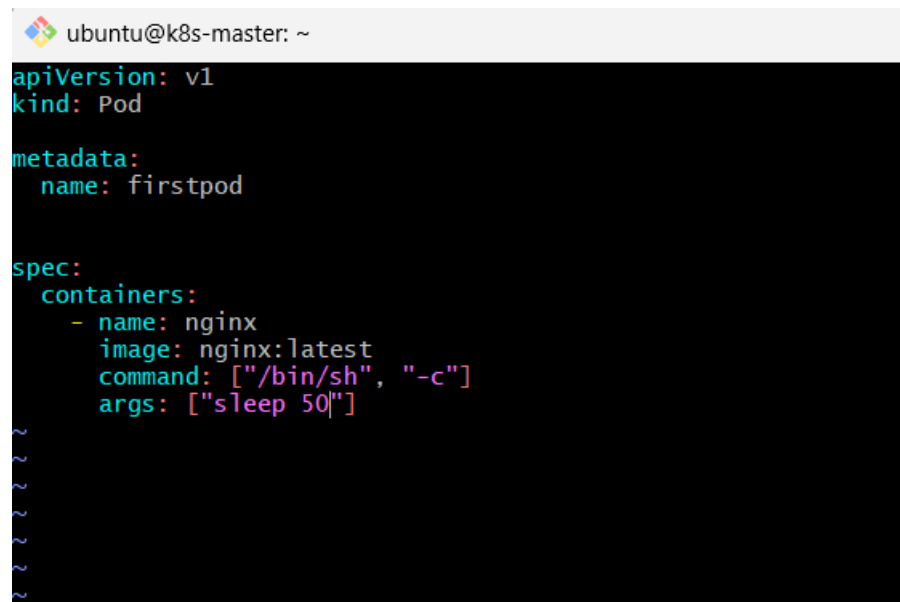
Expected:

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	Ready	control-plane	...	...

```
k8s-worker1  Ready  <none>    ...  ...  
k8s-worker2  Ready  <none>    ...  ...
```

2. Create the YAML File

Open the YAML file:



```
ubuntu@k8s-master: ~  
apiVersion: v1  
kind: Pod  
  
metadata:  
  name: firstpod  
  
spec:  
  containers:  
  - name: nginx  
    image: nginx:latest  
    command: ["/bin/sh", "-c"]  
    args: ["sleep 50"]  
~  
~  
~  
~  
~  
~
```

3. Understand command and args

This is important for your seminar.

command

```
command: ["/bin/sh", "-c"]
```

This tells the container to execute:

```
/bin/sh -c
```

args

```
args: ["sleep 50"]
```

This passes the command:

```
sleep 50
```

So Kubernetes effectively runs:

```
/bin/sh -c "sleep 50"
```

The container will remain running for approximately **50 seconds** and then exit.

4. Save the File

5. Verify the YAML

Run:

```
cat firstpod.yaml
```

6. Deploy the Pod

Run:

```
kubectl apply -f firstpod.yaml
```

Expected:

```
pod/firstpod created
```

7. Check the Pod Status

Immediately run:

```
kubectl get pods
```

You may see:

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

firstpod	1/1	Running	0	5s
----------	-----	---------	---	----

Because the container is sleeping for 50 seconds, it should initially show Running.

8. Describe the Pod

Now run:

```
kubectl describe pod firstpod
```

```
ubuntu@k8s-master: ~  
IPs:  
  IP: 192.168.126.3  
Containers:  
  nginx:  
    Container ID: containerd://7128bf51997f44129d96884e5995f05fafe9383114fe368e706bfd62c1e48c1e  
    Image: nginx:latest  
    Image ID: docker.io/library/nginx@sha256:abe47724e466aeab9a345d8e46a221c2fa8953c7848bb4a3bd9976a7199f8cf2  
    Port: <none>  
    Host Port: <none>  
    Command: /bin/sh  
    -c  
    Args: sleep 50  
    State: Running  
    Started: Wed, 23 Sep 2026 06:50:27 +0000  
    Ready: True  
    Restart Count: 0  
    Environment: <none>  
    Mounts: /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-g5vjh (ro)
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f firstpod.yaml  
pod/firstpod created  
ubuntu@k8s-master:~$ kubectl get pods  
NAME      READY   STATUS    RESTARTS   AGE  
firstpod   1/1     Running   0           8s  
ubuntu@k8s-master:~$ kubectl describe pod firstpod  
Name: firstpod  
Namespace: default  
Priority: 0  
Service Account: default  
Node: k8s-worker2/172.31.26.51  
Start Time: Wed, 23 Sep 2026 06:50:26 +0000  
Labels: <none>  
Annotations: cni.projectcalico.org/containerID: 99f74df80fbfc080c4ef29d1ffe767d40a4de4e705dc2ee3a00e624115a84d1d  
              cni.projectcalico.org/podIP: 192.168.126.3/32  
              cni.projectcalico.org/podIPs: 192.168.126.3/32  
Status: Running  
IP: 192.168.126.3  
IPs:  
  IP: 192.168.126.3  
Containers:  
  nginx:  
    Container ID: containerd://7128bf51997f44129d96884e5995f05fafe9383114fe368e706bfd62c1e48c1e  
    Image: nginx:latest  
    Image ID: docker.io/library/nginx@sha256:abe47724e466aeab9a345d8e46a221c2fa8953c7848bb4a3bd9976a7199f8cf2  
    Port: <none>  
    Host Port: <none>  
    Command: /bin/sh  
    -c  
    Args: sleep 50  
    State: Running  
    Started: Wed, 23 Sep 2026 06:50:27 +0000  
    Ready: True  
    Restart Count: 0
```

9. Check the Pod Again

Run:

kubectl get pod firstpod

Initially:

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

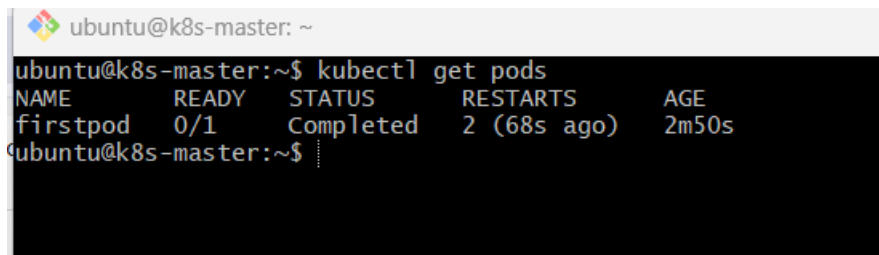
firstpod 1/1 Running 0 20s

After approximately 50 seconds, the container will finish:

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

firstpod	0/1	Completed	0	1m
----------	-----	-----------	---	----

The exact timing can vary slightly.



```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get pods  
NAME      READY   STATUS    RESTARTS   AGE  
firstpod   0/1     Completed  2 (68s ago) 2m50s  
ubuntu@k8s-master:~$
```

10. Check the Container State

Run:

kubectl describe pod firstpod

After the sleep finishes, you may see:

State: Terminated

Reason: Completed

Exit Code: 0

Meaning

Exit Code: 0

means the command completed successfully.

11. Check the Logs

Run:

kubectl logs firstpod

There may be no output because:

sleep 50

doesn't print anything to standard output.

That is normal.

12. Important Difference: command vs args

For your Kubernetes notes:

Field	Purpose
command	Overrides the container image's default ENTRYPOINT
args	Provides arguments to the command
command + args	Together determine what the container executes

4. Create a Pod with Two Containers Task: Create a YAML file to define a Pod with two nginx containers inside. Use kubectl exec to access both containers and verify that both containers can communicate through the same network (e.g., using telnet between them).

Create a Pod with Two Containers

Objective

Create a single Kubernetes Pod containing **two NGINX containers**:

- Container 1: nginx1
- Container 2: nginx2

Both containers will share the **same Pod network namespace**, so they can communicate with each other using localhost.

Important: Containers inside the same Pod share the same IP address and network namespace. Therefore, if both NGINX containers listen on port 80, you cannot bind both to 0.0.0.0:80. For this practical, we will make nginx2 listen on **port 8080**, while nginx1 listens on **port 80**.

1. Check Kubernetes Cluster

First check your nodes:

kubectl get nodes

Expected:

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	Ready	control-plane	...	...
k8s-worker1	Ready	<none>	...	...
k8s-worker2	Ready	<none>	...	...

All required nodes should show:

Ready

2. Create the YAML File

Create: vi twocontainers.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 twocontainers.yaml  
apiVersion: v1  
kind: Pod  
  
metadata:  
  name: twonginx  
  
spec:  
  containers:  
    - name: nginx1  
      image: nginx:latest  
      ports:  
        - containerPort: 80  
    - name: nginx2  
      image: nginx:latest  
      command: ["/bin/sh", "-c"]  
      args:  
        - |  
          sed -i 's/listen      80;/listen      8080;/' /etc/nginx/conf.d/default.conf  
          nginx -g 'daemon off;'  
      ports:  
        - containerPort: 8080
```

3. Understand the YAML

Pod

kind: Pod

metadata:

name: twonginx

This creates a Pod named:

twonginx

First container

- name: nginx1

image: nginx:latest

The first NGINX container listens on:

80

Second container

- name: nginx2

image: nginx:latest

The second NGINX container is configured to listen on:

8080

This avoids a port conflict because both containers share the same network namespace.

4. Verify the YAML

Run:

```
cat twocontainers.yaml
```

Make sure the file looks correct.

5. Create the Pod

Run:

```
kubectl apply -f twocontainers.yaml
```

Expected:

```
pod/twonginx created
```

6. Verify the Pod

Run:

```
kubectl get pods
```

Expected:

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

twonginx 2/2 Running 0 20s

What does 2/2 mean?

It means:

2 containers expected

2 containers running

So both NGINX containers are running.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f twocontainers.yaml  
pod/twonginx created  
ubuntu@k8s-master:~$ kubectl get pods  
NAME      READY   STATUS    RESTARTS   AGE  
firstpod   1/1     Running   6 (2m59s ago)  11m  
twonginx   2/2     Running   0           11s  
ubuntu@k8s-master:~$ |
```

7. See Both Containers

Run:

```
kubectl get pod twonginx -o jsonpath='{.spec.containers[*].name}'
```

Expected:

nginx1 nginx2

You can also run:

```
kubectl describe pod twonginx
```

Look under:

Containers:

You should see:

nginx1

nginx2

```

ubuntu@k8s-master: ~
Image:      nginx:latest
Image ID:   docker.io/library/nginx@sha256:abe47724e466aeab9a345d8e46a221
c2fa8953c7848bb4a3bd9976a7199f8cf2
Port:       8080/TCP
Host Port:  0/TCP
Command:    /bin/sh
-c
Args:       sed -i 's/listen      80;/listen      8080;/' /etc/nginx/conf.d/default.
conf
nginx -g 'daemon off;'

State:      Running
Started:    Wed, 23 Sep 2026 07:01:26 +0000
Ready:      True
Restart Count: 0
Environment: <none>
Mounts:     /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-h4c2q (
ro)
Conditions:
  Type             Status
  PodReadyToStartContainers True
  Initialized       True
  Ready             True
  ContainersReady   True
  PodScheduled      True
Volumes:
  kube-api-access-h4c2q:
    Type:      Projected (a volume that contains injected data fro
    multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional:      false
    DownwardAPI:   true
QoS Class:       BestEffort
Node-Selectors:  <none>
Tolerations:     node.kubernetes.io/not-ready:NoExecute op=Exists fo
r 300s
                node.kubernetes.io/unreachable:NoExecute op=Exists
for 300s
Events:
  Type    Reason      Age   From          Message
  ----    -
  Normal  Scheduled   9m47s default-scheduler Successfully assigned default/two
nginx to k8s-worker1
  Normal  Pulling     9m46s kubelet        Pulling image "nginx:latest"
  Normal  Pulled      9m46s kubelet        Successfully pulled image "nginx:
latest" in 147ms (147ms including waiting). Image size: 63422407 bytes.
  Normal  Created     9m46s kubelet        Created container: nginx1
  Normal  Started     9m46s kubelet        Started container nginx1
  Normal  Pulling     9m46s kubelet        Pulling image "nginx:latest"
  Normal  Pulled      9m46s kubelet        Successfully pulled image "nginx:
latest" in 142ms (142ms including waiting). Image size: 63422407 bytes.
  Normal  Created     9m46s kubelet        Created container: nginx2
  Normal  Started     9m46s kubelet        Started container nginx2

```

8. Check the Pod IP Address

Run:

```
kubectl get pod twonginx -o wide
```

9. Access the First Container

Run:

```
kubectl exec -it twonginx -c nginx1 -- /bin/bash
```

You are now inside nginx1.

Check:

```
hostname
```

Then exit:

```
exit
```

10. Access the Second Container

Run:

```
kubectl exec -it twonginx -c nginx2 -- /bin/bash
```

You are now inside nginx2.

Check:

```
hostname
```

Then exit:

```
exit
```

Both containers should show the **same hostname**, because they are containers inside the same Pod.

11. Test Communication Using localhost

This is the most important part.

Because the two containers share the same network namespace, nginx2 can access nginx1 using:

```
localhost:80
```

From nginx2, run:

```
kubectl exec twonginx -c nginx2 -- curl http://localhost:80
```

You should receive NGINX HTML output such as:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Welcome to nginx!</title>
```

```
...
```

This proves:

nginx2 → localhost:80 → nginx1

```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl get pod twonginx -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE           NOM
INATED NODE   READINESS GATES
twonginx      2/2     Running   0           13m   192.168.194.68 k8s-worker1    <no
ne>          <none>
ubuntu@k8s-master:~$ kubectl exec -it twonginx -c nginx1 -- /bin/bash
root@twonginx:/# hostname
twonginx
root@twonginx:/# exit
exit
ubuntu@k8s-master:~$ kubectl exec -it twonginx -c nginx2 -- /bin/bash
root@twonginx:/# hostname
twonginx
root@twonginx:/# exit
exit
ubuntu@k8s-master:~$ kubectl exec twonginx -c nginx2 -- curl http://localhost:80
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           % Done    0     0     392k      0 --:--:-- --:--:-- --:--:-- 437k
<!DOCTYPE html>
<html>
<head>
<title>welcome to nginx!</title>
<style>
html { color:scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
ubuntu@k8s-master:~$ |

```

12. Test the Second NGINX

Now access port 8080:

```
kubectl exec twonginx -c nginx1 -- curl http://localhost:8080
```

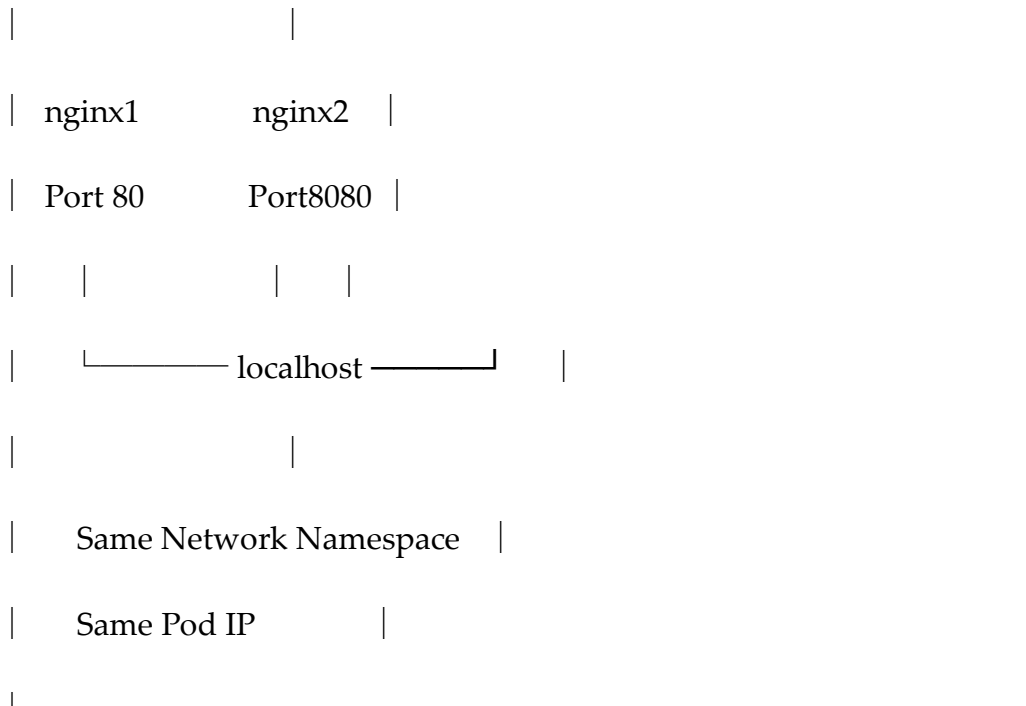
You should again receive NGINX HTML output.

This proves:

nginx1 → localhost:8080 → nginx2

Therefore:





13. Using Telnet

Your task specifically mentions testing with **telnet**.

First check whether telnet exists in nginx1:

```
kubectl exec twonginx -c nginx1 -- which telnet
```

The official NGINX image usually does **not** include telnet.

So if you get no output, that is normal.

Install telnet temporarily

Enter the container:

```
kubectl exec -it twonginx -c nginx1 -- /bin/bash
```

Inside the container:

```
apt-get update
```

```
apt-get install -y telnet
```

Then test nginx2:

```
telnet localhost 8080
```

You should see something similar to:

Trying 127.0.0.1...

Connected to localhost.

This confirms that nginx1 can communicate with nginx2.

Exit telnet:

CTRL +]

Then:

quit

Finally:

exit

5.Set Up an Init Container in a Pod Task: Modify the YAML to include an init container that sleeps for 30 seconds before the main containers start. Verify the init container's execution using kubectl describe pod and check the logs to confirm its completion.

Set Up an Init Container in a Pod

Objective

Create a Pod with:

- **1 Init Container** that sleeps for **30 seconds**
- **1 Main NGINX container**
- Verify that the init container runs **before** the main container.
- Use kubectl describe pod and kubectl logs to confirm the init container completed successfully.

1. Understand Init Containers

An **Init Container** is a special container that runs **before** the main application containers.

The sequence is:

Pod starts

↓

Init Container starts

↓

sleep for 30 seconds

↓

Init Container completes

↓

Main NGINX container starts

↓

Pod becomes Running

If the Init Container fails, Kubernetes does **not** start the main container.

2. Check Your Kubernetes Cluster

Run:

kubectl get nodes

3. Create the YAML File

Create a file:

nano initpod.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: v1  
kind: Pod  
  
metadata:  
  name: initpod  
  
spec:  
  initContainers:  
    - name: init-container  
      image: busybox:latest  
      command: ["/bin/sh", "-c"]  
      args:  
        - echo "Init container started";  
          sleep 30;  
          echo "Init container completed"  
  
  containers:  
    - name: nginx  
      image: nginx:latest
```

4. Understand the YAML

Pod

metadata:

name: initpod

The Pod is named:

initpod

Init Container

initContainers:

- name: init-container

image: busybox:latest

We are using the lightweight busybox image.

The command is:

command: ["/bin/sh", "-c"]

The arguments are:

args:

```
- echo "Init container started";  
  
  sleep 30;  
  
  echo "Init container completed"
```

Therefore, the Init Container performs:

```
echo "Init container started"  
  
sleep 30  
  
echo "Init container completed"
```

Main Container

containers:

```
- name: nginx  
  
  image: nginx:latest
```

The NGINX container starts **only after the Init Container finishes successfully**.

5. Save the YAML

6. Verify the YAML

Run:

```
cat initpod.yaml
```

7. Deploy the Pod

Run:

```
kubectl apply -f initpod.yaml
```

Expected:

```
pod/initpod created
```

8. Immediately Check the Pod

Run:

kubectl get pods

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f initpod.yaml  
pod/initpod created  
ubuntu@k8s-master:~$ kubectl get pods  
NAME          READY   STATUS             RESTARTS   AGE  
firstpod      0/1     CrashLoopBackOff   12 (79s ago) 48m  
initpod       0/1     Init:0/1           0           9s  
twonginx     2/2     Running            0           37m  
ubuntu@k8s-master:~$ |
```

What does Init:0/1 mean?

It means:

0 init containers completed

1 init container exists

The main NGINX container has **not started yet**.

9. Watch the Pod Status

Run:

kubectl get pod initpod -w

10. Describe the Pod

Run:

kubectl describe pod initpod

Look for:

Init Containers:

init-container:

```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl get pod initpod -w
NAME      READY   STATUS    RESTARTS   AGE
initpod   1/1     Running   0           14m
ACubuntu@k8s-master:~$ AC
ubuntu@k8s-master:~$ kubectl describe pod initpod
Name:      initpod
Namespace: default
Priority:   0
Service Account: default
Node:      k8s-worker2/172.31.26.51
Start Time: Wed, 23 Sep 2026 07:38:58 +0000
Labels:    <none>
Annotations: cnf.projectcalico.org/containerID: b4778b381e4ccc3ad774562c409
6596922511a47f186fd90857a4c0eb71c52f5
cnf.projectcalico.org/podIP: 192.168.126.4/32
cnf.projectcalico.org/podIPs: 192.168.126.4/32
Status:    Running
IP:        192.168.126.4
IPs:
  IP: 192.168.126.4
Init Containers:
  init-container:
    Container ID:   containerd://f01adb536125d44f9c8f36bf9da303c211897ce322902e90
da8f109d52768218
    Image:          busybox:latest
    Image ID:       docker.io/library/busybox@sha256:cac8f90bbe42dc962a6b38bb1a2
35948d070385bb9d996bba15a6db8d364008
    Port:           <none>
    Host Port:      <none>
    Command:
      /bin/sh
      -c
    Args:
      echo "Init container started"; sleep 30; echo "Init container completed"
    State:          Terminated
      Reason:       Completed
      Exit Code:    0
      Started:      Wed, 23 Sep 2026 07:38:59 +0000
      Finished:     Wed, 23 Sep 2026 07:39:29 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-hztzv (
ro)
Containers:
  nginx:
    Container ID:   containerd://0f910ff806262bafa340368b39a7fe32f04998a0f35421e
7552ebcc5e4f472c5
    Image:          nginx:latest
    Image ID:       docker.io/library/nginx@sha256:abe47724e466aeab9a345d8e46a22
1c2fa8953c7848bb4a3bd9976a7199f8cf2
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Wed, 23 Sep 2026 07:39:29 +0000
    Ready:          True
    Restart Count:  0

```

11. Check Init Container Logs

This is an important part of the task.

Run:

```
kubectl logs initpod -c init-container
```

Expected:

Init container started

Init container completed

The two messages are separated by approximately **30 seconds**.

This proves that the Init Container executed the sleep command and completed.

12. Check Main NGINX Container

Now run:

```
kubectl get pods
```

Expected:

NAME	READY	STATUS	RESTARTS	AGE
initpod	1/1	Running	0	1m

The 1/1 means the main NGINX container is now running.

13. Check NGINX Logs

Run:

```
kubectl logs initpod -c nginx
```

You should see NGINX startup messages.

For example:

```
/docker-entrypoint.sh: ...
```

...

This confirms that the main container started after the Init Container completed.

14. Check All Containers

Run:

```
kubectl get pod initpod -o jsonpath='{.spec.initContainers[*].name}'
```

Expected:

```
init-container
```

Check the main container:

```
kubectl get pod initpod -o jsonpath='{.spec.containers[*].name}'
```

Expected:

```
nginx
```

15. Verify Init Container Status

You can use:

```
kubectl get pod initpod -o  
jsonpath='{.status.initContainerStatuses[*].state.terminated.reason}'
```

Expected:

Completed

Check the exit code:

```
kubectl get pod initpod -o  
jsonpath='{.status.initContainerStatuses[*].state.terminated.exitCode}'
```

Expected:

0

Exit code 0 means the Init Container completed successfully.

16. Check Pod Events

Run:

```
kubectl describe pod initpod
```

At the bottom, check the **Events** section.

You should see events similar to:

Events:

Normal Pulling Pulling image "busybox:latest"

Normal Pulled Successfully pulled image

Normal Created Created container init-container

Normal Started Started container init-container

Normal Pulling Pulling image "nginx:latest"

Normal Pulled Successfully pulled image

Normal Created Created container nginx

Normal Started Started container nginx

Notice the order:

init-container Started

↓

init-container Completed

↓

nginx Started

17. Important Verification

Run:

```
kubectl describe pod initpod
```

You should find:

Init Containers:

init-container:

State: Terminated

Reason: Completed

Exit Code: 0

Then:

```
kubectl logs initpod -c init-container
```

Expected:

Init container started

Init container completed

And finally:

```
kubectl get pods
```

Expected:

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

initpod	1/1	Running	0	...
---------	-----	---------	---	-----

18. Complete Command Sequence

For your practical/documentation:

1. Check Kubernetes nodes

```
kubectl get nodes
```

2. Create YAML

```
nano initpod.yaml
```

3. Deploy Pod

```
kubectl apply -f initpod.yaml
```

4. Check Pod status

```
kubectl get pods
```

5. Watch Pod startup

```
kubectl get pod initpod -w
```

6. Describe Pod

```
kubectl describe pod initpod
```

7. Check Init Container logs

```
kubectl logs initpod -c init-container
```

8. Check NGINX logs

```
kubectl logs initpod -c nginx
```

9. Check Init Container status

```
kubectl get pod initpod \
```

```
-o jsonpath='{.status.initContainerStatuses[*].state.terminated.reason}'
```

10. Check exit code

```
kubectl get pod initpod \
```

```
-o jsonpath='{.status.initContainerStatuses[*].state.terminated.exitCode}'
```

19. Final YAML

Your final initpod.yaml:

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: initpod
```

```
spec:
```

```
  initContainers:
```

```
    - name: init-container
```

```
      image: busybox:latest
```

```
      command: ["/bin/sh", "-c"]
```

```
      args:
```

```
        - echo "Init container started";
```

```
        sleep 30;
```

```
        echo "Init container completed"
```

containers:

- name: nginx

image: nginx:latest

20. Expected Final Result

Pod status

kubectl get pods

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

initpod	1/1	Running	0	1m
---------	-----	---------	---	----

Init Container logs

kubectl logs initpod -c init-container

Init container started

Init container completed

Init Container description

State: Terminated

Reason: Completed

Exit Code: 0

6.Run a Dry Run Command to Generate YAML Task:

Use the `kubectl run nginx --image=nginx --dry-run=client -o yaml` command to generate a Pod YAML definition. Modify the generated YAML to suit specific requirements (e.g., labels or environment variables) and deploy it.

Run a Dry Run Command to Generate YAML

Objective

Use the `kubectrl run` command with **dry-run** to automatically generate a Kubernetes Pod YAML file. Then modify the generated YAML by adding requirements such as:

- Labels
- Environment variables

Finally, deploy the modified YAML and verify the Pod.

1. Check the Kubernetes Cluster

First verify that your cluster is working:

```
kubectrl get nodes
```

2. Generate YAML Using Dry Run

Run exactly the command from the task:

```
kubectrl run nginx --image=nginx --dry-run=client -o yaml
```

What does this command do?

```
kubectrl run nginx
```

Creates a Pod definition named `nginx`.

```
--image=nginx
```

Uses the NGINX image.

```
--dry-run=client
```

Tells `kubectrl`:

Generate the resource definition locally, but do not create the Pod in Kubernetes.

```
-o yaml
```

Displays the generated definition in YAML format.

3. Expected Output

You should see something similar to:

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl run nginx --image=nginx --dry-run=client -o yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  labels:  
    run: nginx  
    name: nginx  
spec:  
  containers:  
  - image: nginx  
    name: nginx  
    resources: {}  
    dnsPolicy: ClusterFirst  
    restartPolicy: Always  
status: {}  
ubuntu@k8s-master:~$ |
```

4. Save the Generated YAML to a File

Instead of just displaying the YAML, save it directly to a file:

```
kubectl run nginx --image=nginx --dry-run=client -o yaml > nginx-pod.yaml
```

Now check the file:

```
cat nginx-pod.yaml
```

You should see the generated YAML.

```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl run nginx --image=nginx --dry-run=client -o yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: nginx
    name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
ubuntu@k8s-master:~$ kubectl run nginx --image=nginx --dry-run=client -o yaml >
nginx-pod.yaml
ubuntu@k8s-master:~$ cat nginx-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: nginx
    name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
ubuntu@k8s-master:~$ |

```

5. Modify the YAML

Open the file:

nano nginx-pod.yaml

For this practical, let's add:

Label

app: nginx

Environment variables

myname: raji

City: Hyderabad

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: v1  
kind: Pod  
metadata:  
  labels:  
    run: nginx  
    app: nginx  
    env: test  
    name: nginx  
spec:  
  containers:  
  - image: nginx  
    name: nginx  
    env:  
    - name: myname  
      value: "raji"  
    - name: City  
      value: "Chirala"  
    resources: {}  
  dnsPolicy: ClusterFirst  
  restartPolicy: Always  
status: {}
```

6. Understand the Modifications

Labels

We added:

labels:

app: nginx

environment: learning

Labels help identify and organize Kubernetes resources.

For example:

```
kubectl get pods --show-labels
```

You should see something similar to:

NAME	READY	STATUS	RESTARTS	AGE	LABELS
nginx	1/1	Running	0	20s	app=nginx,environment=learning

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f nginx-pod.yaml  
pod/nginx created  
ubuntu@k8s-master:~$ kubectl get pod nginx --show-labels  
NAME      READY   STATUS    RESTARTS   AGE   LABELS  
nginx     1/1     Running   0           10s   app=nginx,environment=learning  
ubuntu@k8s-master:~$
```

7. Validate the YAML

Before deploying, you can use:

```
kubectl apply --dry-run=client -f nginx-pod.yaml
```

Expected:

```
pod/nginx created (dry run)
```

This verifies the YAML without actually creating the Pod.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f nginx-pod.yaml  
pod/nginx created  
ubuntu@k8s-master:~$ kubectl get pod nginx --show-labels  
NAME      READY   STATUS    RESTARTS   AGE   LABELS  
nginx     1/1     Running   0           10s   app=nginx,environment=learning  
ubuntu@k8s-master:~$ kubectl apply --dry-run=client -f nginx-pod.yaml  
pod/nginx configured (dry run)  
ubuntu@k8s-master:~$
```

8. Deploy the Pod

Now deploy it:

```
kubectl apply -f nginx-pod.yaml
```

Expected:

```
pod/nginx created
```

9. Verify the Pod

Run:

```
kubectl get pods
```

Expected:

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

nginx	1/1	Running	0	10s
-------	-----	---------	---	-----

10. Verify the Labels

Run:

```
kubectl get pod nginx --show-labels
```

Expected:

NAME	READY	STATUS	RESTARTS	AGE	LABELS
------	-------	--------	----------	-----	--------

nginx	1/1	Running	0	...	app=nginx,environment=learning
-------	-----	---------	---	-----	--------------------------------

You can also use:

```
kubectl get pod nginx -o jsonpath='{.metadata.labels}'
```

11. Verify Environment Variables

Run:

```
kubectl exec nginx -- env
```

Look for:

myname=sabair

City=Hyderabad

For a cleaner result:

```
kubectl exec nginx -- env | grep -E 'myname|City'
```

Expected:

myname=sabair

City=Hyderabad

12. Verify the YAML Configuration

Run:

```
kubectl describe pod nginx
```

Under the container section, you should find:

Containers:

nginx:

Image: nginx

Environment:

myname: sabair

City: Hyderabad

Under the Pod metadata you should find the labels:

Labels:

app: nginx

environment: learning

13. Check the Generated/Deployed YAML

You can see the actual configuration stored in Kubernetes:

```
kubectl get pod nginx -o yaml
```

This displays the complete Pod definition, including Kubernetes-generated fields.

14. Important Difference: Dry Run vs Actual Deployment

Dry run

```
kubectl run nginx --image=nginx --dry-run=client -o yaml
```

This **does not create** the Pod.

It only generates the YAML.

Save to file

```
kubectl run nginx --image=nginx --dry-run=client -o yaml > nginx-pod.yaml
```

This generates and saves the YAML.

Deploy

```
kubectl apply -f nginx-pod.yaml
```

This actually creates the Pod.

7. Use kubectl apply vs kubectl create Task: Create a YAML file to define a Pod. First, deploy it using kubectl create -f <file_name>.yaml and then modify the YAML (e.g., change the image version). Use kubectl apply to redeploy and verify the difference between both commands.

kubectl create vs kubectl apply

Objective

Create a Pod using a YAML file and understand the difference between:

```
kubectl create -f pod.yaml
```

and:

```
kubectl apply -f pod.yaml
```

We will:

1. Create a Pod with NGINX.
2. Deploy it using kubectl create.
3. Modify the NGINX image version.
4. Try kubectl create again and observe what happens.
5. Use kubectl apply to update the Pod.
6. Verify the difference.

1. Prerequisites

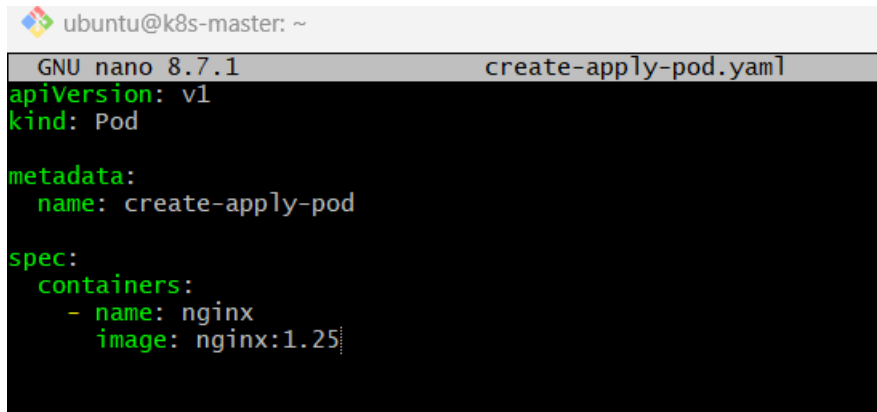
Check your Kubernetes cluster:

kubectl get nodes

2. Create the Initial YAML File

Create a file:

nano create-apply-pod.yaml



```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 create-apply-pod.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: create-apply-pod  
spec:  
  containers:  
  - name: nginx  
    image: nginx:1.25
```

3. Verify the YAML

Run:

cat create-apply-pod.yaml

4. Deploy Using kubectl create

Now use:

kubectl create -f create-apply-pod.yaml

Expected:

pod/create-apply-pod created

This creates the Pod.

5. Verify the Pod

Run:

kubectl get pods

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get pods  
NAME          READY   STATUS    RESTARTS   AGE  
create-apply-pod 1/1     Running   0           10m  
firstpod       0/1     CrashLoopBackOff 22 (4m ago) 110m  
initpod        1/1     Running   0           61m  
nginx          1/1     Running   0           21m  
twonginx       2/2     Running   0           99m  
ubuntu@k8s-master:~$ |
```

6. Check the NGINX Image Version

Run:

```
kubectl get pod create-apply-pod -o jsonpath='{.spec.containers[0].image}'
```

Expected:

nginx:1.25

You can also use:

```
kubectl describe pod create-apply-pod
```

Look for:

Containers:

nginx:

Image: nginx:1.25

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get pods  
NAME          READY   STATUS    RESTARTS   AGE  
create-apply-pod 1/1     Running   0           10m  
firstpod       0/1     CrashLoopBackOff 22 (4m ago) 110m  
initpod        1/1     Running   0           61m  
nginx          1/1     Running   0           21m  
twonginx       2/2     Running   0           99m  
ubuntu@k8s-master:~$ kubectl get pod create-apply-pod -o jsonpath='{.spec.containers[0].image}'  
nginx:1.25ubuntu@k8s-master:~$
```

7. Modify the YAML

Now edit the YAML:

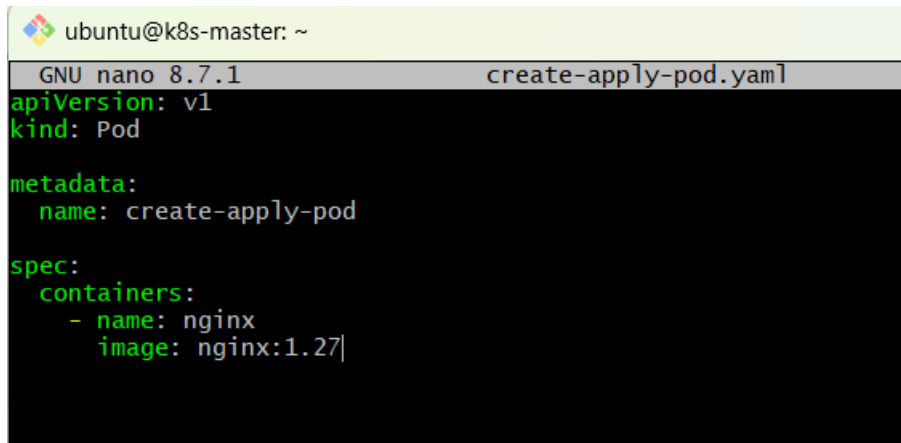
```
nano create-apply-pod.yaml
```

Change:

image: nginx:1.25

to:

image: nginx:1.27



```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 create-apply-pod.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: create-apply-pod  
spec:  
  containers:  
  - name: nginx  
    image: nginx:1.27|
```

8. Try kubectl create Again

Now run:

```
kubectl create -f create-apply-pod.yaml
```

You will get an error similar to:

Error from server (AlreadyExists): pods "create-apply-pod" already exists

Why?

Because kubectl create is intended to **create a new resource**.

The Pod already exists.

Therefore:

Existing Pod

↓

kubectl create

↓

AlreadyExists error

create does not update the existing Pod.

9. Check the Image

Run:

```
kubectl get pod create-apply-pod -o jsonpath='{.spec.containers[0].image}'
```

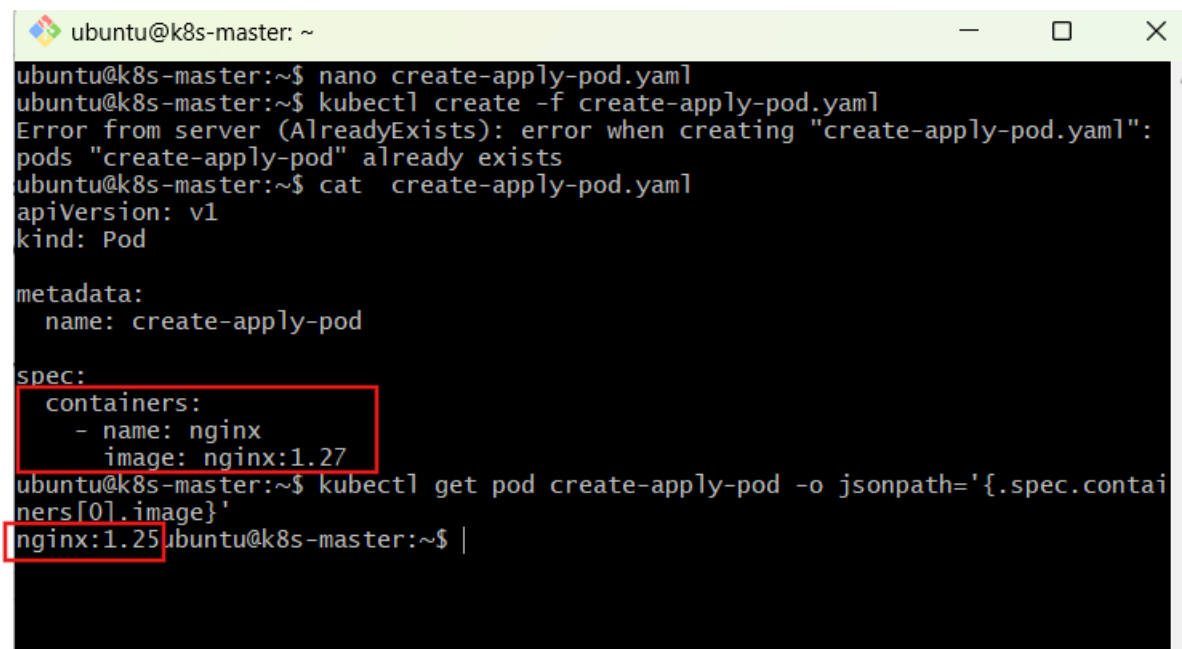
It will still show:

```
nginx:1.25
```

Even though your YAML contains:

```
nginx:1.27
```

The running Pod has not been changed.

A terminal window titled 'ubuntu@k8s-master: ~' with standard window controls. The terminal shows the following commands and output:

```
ubuntu@k8s-master:~$ nano create-apply-pod.yaml
ubuntu@k8s-master:~$ kubectl create -f create-apply-pod.yaml
Error from server (AlreadyExists): error when creating "create-apply-pod.yaml":
pods "create-apply-pod" already exists
ubuntu@k8s-master:~$ cat create-apply-pod.yaml
apiVersion: v1
kind: Pod

metadata:
  name: create-apply-pod

spec:
  containers:
  - name: nginx
    image: nginx:1.27
ubuntu@k8s-master:~$ kubectl get pod create-apply-pod -o jsonpath='{.spec.containers[0].image}'
nginx:1.25
ubuntu@k8s-master:~$ |
```

Red boxes highlight the 'containers' section in the YAML file and the output 'nginx:1.25' of the kubectl command.

10. Now Use kubectl apply

Run:

```
kubectl apply -f create-apply-pod.yaml
```

You may receive an error for a direct Pod image update because **many Pod fields are immutable after creation**. A typical result is an error indicating that the Pod specification cannot be changed.

This is an important Kubernetes point:

`kubectl apply` can update resources, but an existing Pod's container image is not generally updated in-place through a normal Pod manifest.

So, for a **Pod-specific demonstration**, use `apply` with a field that can be updated, or delete/recreate the Pod.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f create-apply-pod.yaml  
Warning: resource pods/create-apply-pod is missing the kubect.kubernetes.io/last-applied-configuration annotation which is required by kubectl apply. kubectl apply should only be used on resources created declaratively by either kubectl create --save-config or kubectl apply. The missing annotation will be patched automatically.  
pod/create-apply-pod configured  
ubuntu@k8s-master:~$
```

11. Best Way to Demonstrate create vs apply

For a clean practical demonstration, use a **Deployment** rather than a standalone Pod when changing the container image.

However, since your task specifically asks for a **Pod**, you can demonstrate the difference this way:

Initial creation

```
kubectl create -f create-apply-pod.yaml
```

Creates the Pod.

Modify YAML

Change:

```
image: nginx:1.25
```

to:

```
image: nginx:1.27
```

create

```
kubectl create -f create-apply-pod.yaml
```

Result:

AlreadyExists

apply

```
kubectl apply -f create-apply-pod.yaml
```

Result: Kubernetes attempts to reconcile the YAML, but the immutable Pod fields prevent this particular image change from being applied directly.

12. Correct Practical Demonstration Using a Deployment

If your instructor specifically wants to **show that apply can modify the image**, use a Deployment.

Create:

```
nano nginx-deployment.yaml
```

Initial YAML:

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: nginx-deployment
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: nginx
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: nginx
```

spec:

containers:

- name: nginx

image: nginx:1.25

13. Create the Deployment

Run:

```
kubectl create -f nginx-deployment.yaml
```

Expected:

```
deployment.apps/nginx-deployment created
```

Check:

```
kubectl get deployments
```

Expected:

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx-deployment	1/1	1	1	...

Check the Pod:

```
kubectl get pods
```

14. Modify the Image

Edit:

```
nano nginx-deployment.yaml
```

Change:

```
image: nginx:1.25
```

to:

```
image: nginx:1.27
```

15. Try create

Run:

```
kubectl create -f nginx-deployment.yaml
```

You will get:

Error from server (AlreadyExists): deployments.apps "nginx-deployment" already exists

Because the Deployment already exists.

16. Use apply

Now run:

```
kubectl apply -f nginx-deployment.yaml
```

Expected:

deployment.apps/nginx-deployment configured

This time the existing Deployment is updated.

Kubernetes creates a new ReplicaSet/Pod with the new image.

17. Verify the New Image

Run:

```
kubectl get deployment nginx-deployment -o  
jsonpath='{.spec.template.spec.containers[0].image}'
```

Expected:

nginx:1.27

Check the Pods:

```
kubectl get pods
```

You should see the new Pod running.

18. Simple Difference

Command	Purpose	Existing resource
kubectl create -f file.yaml	Create a new resource	Returns AlreadyExists
kubectl apply -f file.yaml	Create or reconcile/update declarative configuration	Updates supported fields
kubectl create	Create only	Does not update
kubectl apply	Create + update	Designed for declarative management

8.Edit an Existing Pod Configuration Task: Use kubectl edit pod <pod_name> to modify the running Pod's environment variables or image. After making the changes, verify if they took effect by checking the container logs or environment variables using kubectl exec.

Edit an Existing Pod Configuration Using kubectl edit

Objective

To learn how to modify an existing Kubernetes Pod using:

kubectl edit pod <pod_name>

We will:

1. Create a Pod with an environment variable.
2. Use kubectl edit to modify the environment variable.
3. Verify the change using kubectl exec.
4. Also understand what happens when trying to change the **image** of an existing Pod.

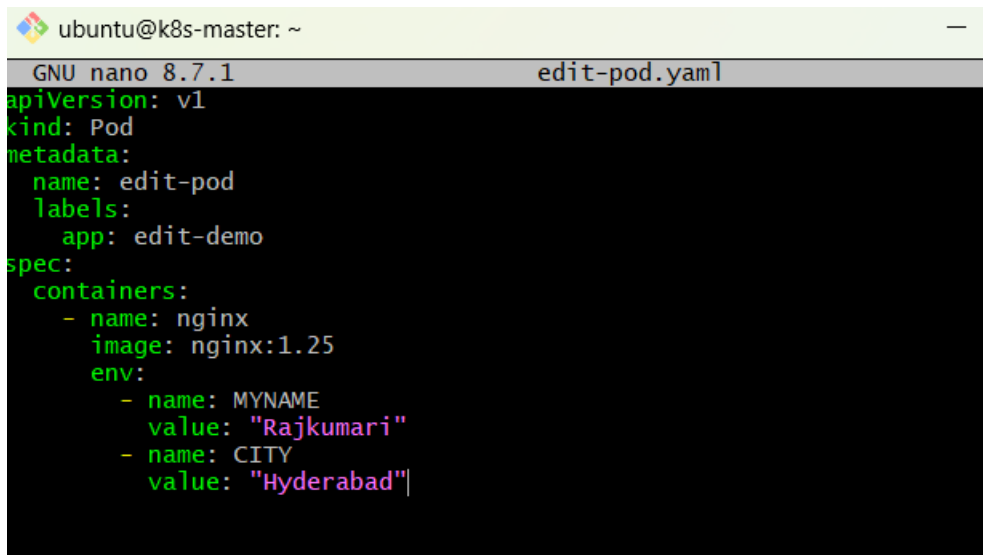
Important: A standalone Pod has many immutable fields. In particular, changing the container image of an existing Pod with kubectl edit normally will **not** work because

Kubernetes does not allow changing most Pod spec fields after creation. For image changes, a **Deployment** is the recommended approach.

Part 1: Create the Pod

Create a YAML file:

vi edit-pod.yaml



```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 edit-pod.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: edit-pod  
  labels:  
    app: edit-demo  
spec:  
  containers:  
    - name: nginx  
      image: nginx:1.25  
      env:  
        - name: MYNAME  
          value: "Rajkumari"  
        - name: CITY  
          value: "Hyderabad"
```

Step 2: Create the Pod

Run:

kubectl create -f edit-pod.yaml

Expected:

pod/edit-pod created

Check the Pod:

kubectl get pod edit-pod

Expected:

NAME	READY	STATUS	RESTARTS	AGE
edit-pod	1/1	Running	0	...

Step 3: Check the Existing Environment Variables

Run:

```
kubectl exec edit-pod -- env
```

You should see:

```
MYNAME=Rajkumari
```

```
CITY=Hyderabad
```

You can also check only MYNAME:

```
kubectl exec edit-pod -- printenv MYNAME
```

Output:

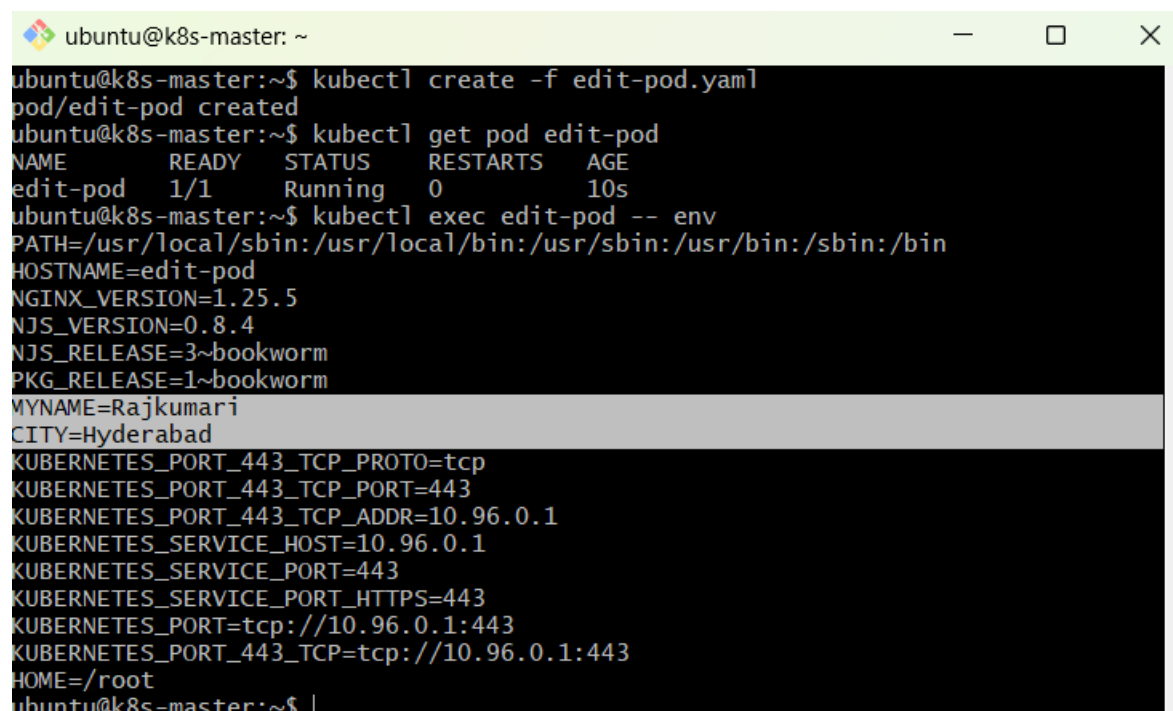
```
Rajkumari
```

Check CITY:

```
kubectl exec edit-pod -- printenv CITY
```

Output:

```
Hyderabad
```



```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl create -f edit-pod.yaml  
pod/edit-pod created  
ubuntu@k8s-master:~$ kubectl get pod edit-pod  
NAME      READY   STATUS    RESTARTS   AGE  
edit-pod  1/1     Running   0           10s  
ubuntu@k8s-master:~$ kubectl exec edit-pod -- env  
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin  
HOSTNAME=edit-pod  
NGINX_VERSION=1.25.5  
NJS_VERSION=0.8.4  
NJS_RELEASE=3~bookworm  
PKG_RELEASE=1~bookworm  
MYNAME=Rajkumari  
CITY=Hyderabad  
KUBERNETES_PORT_443_TCP_PROTO=tcp  
KUBERNETES_PORT_443_TCP_PORT=443  
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1  
KUBERNETES_SERVICE_HOST=10.96.0.1  
KUBERNETES_SERVICE_PORT=443  
KUBERNETES_SERVICE_PORT_HTTPS=443  
KUBERNETES_PORT=tcp://10.96.0.1:443  
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443  
HOME=/root  
ubuntu@k8s-master:~$ |
```

Step 4: Edit the Existing Pod

Now run:

```
kubectl edit pod edit-pod
```

Kubernetes will open the Pod definition in your editor.

Find:

env:

- name: MYNAME

- value: "Rajkumari"

- name: CITY

- value: "Hyderabad"

Change:

- value: "Rajkumari"

to:

- value: "Ricky"

So it becomes:

env:

- name: MYNAME

- value: "Ricky"

- name: CITY

- value: "Hyderabad"

Save and exit.

```
ubuntu@k8s-master: ~  
uid: 9a88419a-5027-4956-b6b4-b0bf38301484  
spec:  
  containers:  
  - env:  
    - name: MYNAME  
      value: Rickyraji  
    - name: CITY  
      value: Hyderabad  
    image: nginx:1.25  
    imagePullPolicy: IfNotPresent  
    name: nginx  
    resources: {}  
    terminationMessagePath: /dev/termination-log  
    terminationMessagePolicy: File  
    volumeMounts:  
    - mountPath: /var/run/secrets/kubernetes.io/serviceaccount  
      name: kube-api-access-v92h4  
      readOnly: true  
  dnsPolicy: ClusterFirst  
  enableServiceLinks: true  
  nodeName: k8s-worker1  
  preemptionPolicy: PreemptLowerPriority  
  priority: 0  
:wqll
```

Step 5: What Will Happen?

This is the important part.

For a **running standalone Pod**, Kubernetes generally does **not allow changing existing container environment variables in place**.

You may receive an error similar to:

The Pod "edit-pod" is invalid:

spec: Forbidden: pod updates may not change fields other than ...

So if this happens, **it is expected behavior**.

The important Kubernetes concept is:

kubectll edit allows editing the resource

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl edit pod edit-pod  
error: pods "edit-pod" is invalid  
A copy of your changes has been stored to "/tmp/kubectl-edit-1619025955.yaml"  
error: Edit cancelled, no valid changes were saved.  
ubuntu@k8s-master:~$
```

Step 6: Verify the Environment Variable

Run:

```
kubectl exec edit-pod -- printenv MYNAME
```

If the edit was rejected, you will still see:

Rajkumari

This proves that the environment variable was **not changed** in the running Pod.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl edit pod edit-pod  
error: pods "edit-pod" is invalid  
A copy of your changes has been stored to "/tmp/kubectl-edit-1619025955.yaml"  
error: Edit cancelled, no valid changes were saved.  
ubuntu@k8s-master:~$ kubectl exec edit-pod -- printenv MYNAME  
Rajkumari  
ubuntu@k8s-master:~$
```

Part 2: Try Editing the Image

Now run:

```
kubectl edit pod edit-pod
```

Find:

containers:

- name: nginx

- image: nginx:1.25

Change it to:

containers:

- name: nginx

- image: nginx:1.27

Save and exit.

You will normally get an immutability error similar to:

spec: Forbidden: pod updates may not change fields other than ...

Therefore, a standalone Pod is **not the correct resource for demonstrating image updates with kubectl edit**.

```
ubuntu@k8s-master: ~  
namespace: default  
resourceVersion: "27341"  
uid: 9a88419a-5027-4956-b6b4-b0bf38301484  
spec:  
  containers:  
  - env:  
    - name: MYNAME  
      value: Rajkumari  
    - name: CITY  
      value: Hyderabad  
    image: nginx:1.27  
    imagePullPolicy: IfNotPresent  
    name: nginx  
    resources: {}  
    terminationMessagePath: /dev/termination-log  
    terminationMessagePolicy: File  
    volumeMounts:  
    - mountPath: /var/run/secrets/kubernetes.io/serviceaccount  
      name: kube-api-access-v92h4  
      readOnly: true  
  dnsPolicy: ClusterFirst  
  enableServiceLinks: true  
  nodeName: k8s-worker1  
:wq!!
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl edit pod edit-pod  
pod/edit-pod edited  
ubuntu@k8s-master:~$ cat edit-pod.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: edit-pod  
  labels:  
    app: edit-demo  
spec:  
  containers:  
  - name: nginx  
    image: nginx:1.25  
    env:  
    - name: MYNAME  
      value: "Rajkumari"  
    - name: CITY  
      value: "Hyderabad"  
ubuntu@k8s-master:~$ |
```

Part 3: Recommended Way — Use a Deployment

For real Kubernetes practice, use a Deployment when you want to change an image or configuration.

Create:

vi edit-deployment.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: edit-deployment  
spec:  
  replicas: 1  
  selector:  
    matchLabels:  
      app: edit-demo  
  template:  
    metadata:  
      labels:  
        app: edit-demo  
    spec:  
      containers:  
      - name: nginx  
        image: nginx:1.25  
        env:  
        - name: MYNAME  
          value: "Rajkumari"  
        - name: CITY  
          value: "Hyderabad"
```

Create it:

```
kubectl create -f edit-deployment.yaml
```

Check:

```
kubectl get deployment
```

Expected:

NAME	READY	UP-TO-DATE	AVAILABLE
edit-deployment	1/1	1	1

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano edit-deployment.yaml  
ubuntu@k8s-master:~$ kubectl create -f edit-deployment.yaml  
deployment.apps/edit-deployment created  
ubuntu@k8s-master:~$ kubectl get deployment  
NAME                READY    UP-TO-DATE    AVAILABLE    AGE  
edit-deployment     1/1      1             1            12s  
ubuntu@k8s-master:~$ |
```

Step 4: Edit the Deployment

Run:

```
kubectl edit deployment edit-deployment
```

Find:

```
image: nginx:1.25
```

Change it to:

```
image: nginx:1.27
```

Save and exit.

The Deployment will create a new Pod with the new image.

Check:

```
kubectl get pods
```

You should see the new Pod running.

```
ubuntu@k8s-master: ~  
  app: edit-demo  
  strategy:  
    rollingUpdate:  
      maxSurge: 25%  
      maxUnavailable: 25%  
    type: RollingUpdate  
  template:  
    metadata:  
      labels:  
        app: edit-demo  
    spec:  
      containers:  
      - env:  
        - name: MYNAME  
          value: Rajkumari  
        - name: CITY  
          value: Hyderabad  
        image: nginx:1.27  
        imagePullPolicy: IfNotPresent  
        name: nginx  
        resources: {}  
        terminationMessagePath: /dev/termination-log  
        terminationMessagePolicy: File  
:wq !  
  
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl edit deployment edit-deployment  
deployment.apps/edit-deployment edited  
ubuntu@k8s-master:~$ kubectl get pods  
NAME                                READY   STATUS              RESTARTS   AGE  
create-apply-pod                    1/1     Running             1 (37m ago) 57m  
edit-deployment-89c465f4f-w6hfq     1/1     Running             0           13s  
edit-pod                             1/1     Running             1 (15m ago) 24m  
firstpod                             0/1     CrashLoopBackOff    30 (3m2s ago) 157  
m  
initpod                             1/1     Running             0           108  
m  
nginx                               1/1     Running             0           68m  
twonginx                             2/2     Running             0           146  
m  
ubuntu@k8s-master:~$ kubectl get pods  
NAME                                READY   STATUS              RESTARTS   AGE  
create-apply-pod                    1/1     Running             1 (38m ago) 57m  
edit-deployment-89c465f4f-w6hfq     1/1     Running             0           29s  
edit-pod                             1/1     Running             1 (15m ago) 24m  
firstpod                             0/1     CrashLoopBackOff    30 (3m18s ago) 157m  
initpod                             1/1     Running             0           109m  
nginx                               1/1     Running             0           69m  
twonginx                             2/2     Running             0           146m  
ubuntu@k8s-master:~$ |
```

Step 5: Verify the Image

Run:

```
kubectl get deployment edit-deployment -o  
jsonpath='{.spec.template.spec.containers[0].image}'
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get deployment edit-deployment -o jsonpath='{.spec.template.spec.containers[0].image}'  
nginx:1.27  
ubuntu@k8s-master:~$ kubectl get pods -o wide  
NAME                                READY   STATUS              RESTARTS   AGE   IP              NODE           NOMINATED NODE   READINESS GATES  
create-apply-pod                    1/1     Running             1 (40m ago) 59m   192.168.126.5   k8s-worker2    <none>            <none>  
edit-deployment-89c465f4f-w6hfq     1/1     Running             0       2m24s  192.168.126.7   k8s-worker2    <none>            <none>  
edit-pod                             1/1     Running             1 (17m ago) 26m   192.168.194.70  k8s-worker1    <none>            <none>  
firstpod                             0/1     CrashLoopBackOff    30 (5m13s ago) 159m  192.168.126.3   k8s-worker2    <none>            <none>  
initpod                             1/1     Running             0       110m   192.168.126.4   k8s-worker2    <none>            <none>  
nginx                               1/1     Running             0       70m   192.168.194.69  k8s-worker1    <none>            <none>  
twonginx                             2/2     Running             0       148m   192.168.194.68  k8s-worker1    <none>            <none>  
ubuntu@k8s-master:~$ |
```

Step 6: Verify Environment Variables

For the Deployment Pod:

```
kubectl exec <pod-name> -- printenv MYNAME
```

Expected:

Rajkumari

And:

```
kubectl exec <pod-name> -- printenv CITY
```

Expected:

Hyderabad

Why?

A **Pod** is intended to be a relatively immutable running unit.

A **Deployment** manages Pods and can create replacement Pods whenever the Pod template changes

9. Expose a Pod Using a Service Task: Create a YAML file to expose your firstpod using a Service (ClusterIP). Ensure that your service is exposing the Pod on port 80 and verify it using `kubectl get svc`.

Expose a Pod Using a Service — ClusterIP

Objective

To expose the existing firstpod using a Kubernetes **Service of type ClusterIP** and make the Pod accessible on **port 80** within the Kubernetes cluster.

Prerequisites

Before starting, make sure:

- Kubernetes cluster is running.
- firstpod already exists.
- firstpod is in Running state.

- kubectl is configured correctly.

Check:

```
kubectl get pods
```

Expected:

NAME	READY	STATUS	RESTARTS	AGE
firstpod	1/1	Running	0	...

Step 1: Check the Pod Labels

A Service selects Pods using **labels**, so first check the labels of firstpod.

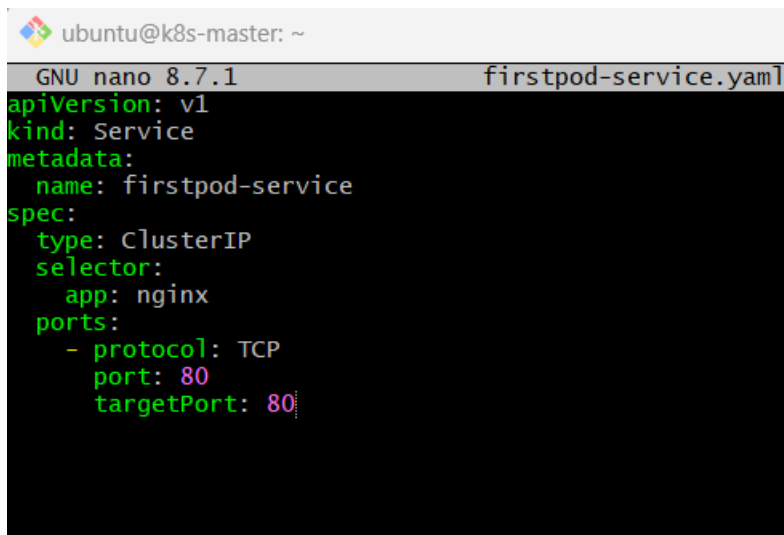
Run:

```
kubectl get pod firstpod --show-labels
```

Step 2: Create the Service YAML File

Create a file:

```
vi firstpod-service.yaml
```



```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 firstpod-service.yaml  
apiVersion: v1  
kind: Service  
metadata:  
  name: firstpod-service  
spec:  
  type: ClusterIP  
  selector:  
    app: nginx  
  ports:  
    - protocol: TCP  
      port: 80  
      targetPort: 80
```

Step 3: Understand the YAML

apiVersion

apiVersion: v1

Service belongs to the Kubernetes core API group.

kind

kind: Service

We are creating a Kubernetes Service.

name

name: firstpod-service

The name of our Service.

type

type: ClusterIP

ClusterIP makes the Service accessible **inside the Kubernetes cluster**.

It does not expose the application directly to the Internet.

selector

selector:

app: nginx

This tells Kubernetes:

Send Service traffic to Pods having the label app=nginx.

port

port: 80

This is the port exposed by the Service.

targetPort

targetPort: 80

This is the port on the selected Pod/container where traffic is sent.

So the flow is:

Client inside cluster

|

| port 80

v

firstpod-service

ClusterIP

|

| targetPort 80

v

firstpod

nginx

port 80

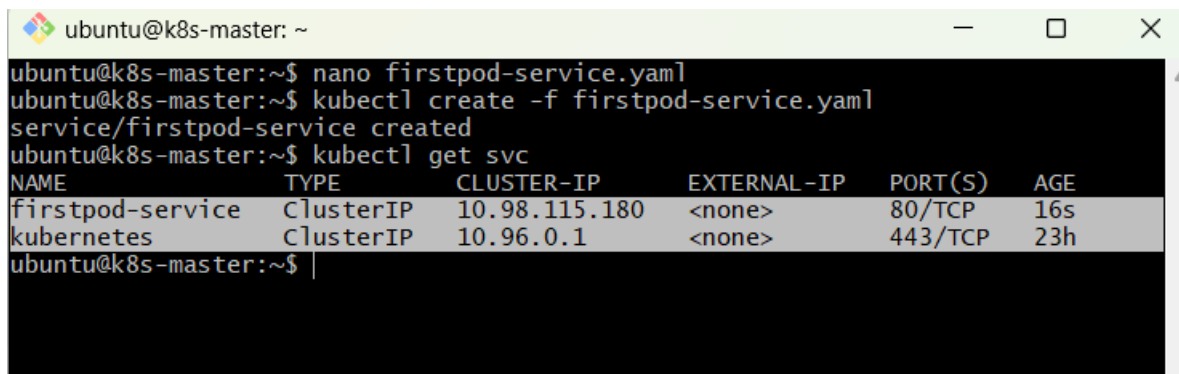
Step 4: Create the Service

Run:

kubectl create -f firstpod-service.yaml

Expected:

service/firstpod-service created



```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano firstpod-service.yaml  
ubuntu@k8s-master:~$ kubectl create -f firstpod-service.yaml  
service/firstpod-service created  
ubuntu@k8s-master:~$ kubectl get svc  
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE  
firstpod-service    ClusterIP   10.98.115.180 <none>       80/TCP     16s  
kubernetes           ClusterIP   10.96.0.1     <none>       443/TCP    23h  
ubuntu@k8s-master:~$
```

Step 6: Get Detailed Service Information

Run:

kubectl describe svc firstpod-service

```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl describe svc firstpod-service
Name: firstpod-service
Namespace: default
Labels: <none>
Annotations: <none>
Selector: app=nginx
Type: ClusterIP
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.98.115.180
IPs: 10.98.115.180
Port: <unset> 80/TCP
TargetPort: 80/TCP
Endpoints: 192.168.194.69:80
Session Affinity: None
Internal Traffic Policy: Cluster
Events: <none>
ubuntu@k8s-master:~$ |

```

Step 7: Verify the Service Endpoints

Run:

```
kubectl get endpoints firstpod-service
```

Expected:

NAME	ENDPOINTS	AGE
firstpod-service	10.244.0.5:80	1m

The actual IP will be different in your cluster.

This confirms that the Service is connected to firstpod.

Step 8: Verify Using EndpointSlices

Modern Kubernetes also uses EndpointSlices.

Run:

```
kubectl get endpointslice
```

Step 9: Test the Service From Inside the Cluster

Because ClusterIP is internal to the cluster, test it from another Pod.

First get the Service IP:

```
kubectl get svc firstpod-service
```

For example:

CLUSTER-IP

10.96.123.45

Run a temporary test Pod:

```
kubectl run test-pod --image=curlimages/curl --rm -it -- sh
```

Inside the temporary Pod:

```
curl http://firstpod-service:80
```

You should receive the NGINX welcome page HTML.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get svc firstpod-service  
NAME          TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE  
firstpod-service ClusterIP   10.98.115.180 <none>       80/TCP     5m49s  
ubuntu@k8s-master:~$ kubectl run test-pod --image=curlimages/curl --rm -it -- sh  
All commands and output from this session will be recorded in container logs, in  
cluding credentials and sensitive information passed through the command prompt.  
If you don't see a command prompt, try pressing enter.  
~ $  
~ $  
~ $ curl http://firstpod-service:80  
<!DOCTYPE html>  
<html>  
<head>  
<title>Welcome to nginx!</title>  
<style>  
html { color-scheme: light dark; }  
body { width: 35em; margin: 0 auto;  
font-family: Tahoma, Verdana, Arial, sans-serif; }  
</style>  
</head>  
<body>  
<h1>welcome to nginx!</h1>  
<p>If you see this page, nginx is successfully installed and working.  
Further configuration is required for the web server, reverse proxy,  
API gateway, load balancer, content cache, or other features.</p>  
  
<p>For online documentation and support please refer to  
<a href="https://nginx.org/">nginx.org</a>.<br/>  
To engage with the community please visit  
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>  
For enterprise grade support, professional services, additional  
security features and capabilities please refer to  
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>  
  
<p><em>Thank you for using nginx.</em></p>  
</body>  
</html>  
~ $ |
```

10.Pod with Resource Limits and Requests

Task: Add resource requests and limits to the containers in your YAML file. Specify CPU

and memory requests/limits for both containers and deploy the Pod. Use `kubectl describe pod` to verify if the resource configurations are correctly applied.

od with Resource Requests and Limits

Objective

To configure **CPU and memory requests and limits** for both containers in a Kubernetes Pod and verify the configuration using:

`kubectl describe pod`

We will create one Pod containing **two containers** and assign resources to each container.

1. Prerequisites

Make sure:

- Kubernetes cluster is running.
- `kubectl` is configured.
- Master/control-plane and worker nodes are in Ready state.

Check:

`kubectl get nodes`

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: v1  
kind: Pod  
metadata:  
  name: resource-pod  
  labels:  
    app: resource-demo  
spec:  
  containers:  
    - name: nginx1  
      image: nginx:latest  
      resources:  
        requests:  
          cpu: "100m"  
          memory: "128Mi"  
        limits:  
          cpu: "200m"  
          memory: "256Mi"  
    - name: nginx2  
      image: nginx:latest  
      resources:  
        requests:  
          cpu: "200m"  
          memory: "256Mi"  
        limits:  
          cpu: "400m"  
          memory: "512Mi"
```

3. Understand the Resource Configuration

For the first container:

resources:

requests:

cpu: "100m"

memory: "128Mi"

limits:

cpu: "200m"

memory: "256Mi"

CPU Request

100m

m means **millicpu**.

1000m = 1 CPU

100m = 0.1 CPU

200m = 0.2 CPU

So nginx1 requests:

0.1 CPU

and can use up to:

0.2 CPU

Memory Request

128Mi

This means the container requests approximately **128 MiB of memory**.

CPU Limit

200m

The container can use up to **0.2 CPU**.

Memory Limit

256Mi

The container can use up to **256 MiB**.

4. Resource Configuration for Container 2

For nginx2:

resources:

requests:

cpu: "200m"

memory: "256Mi"

limits:

cpu: "400m"

memory: "512Mi"

Therefore:

	Container CPU Request	CPU Limit	Memory Request	Memory Limit
nginx1	100m	200m	128Mi	256Mi
nginx2	200m	400m	256Mi	512Mi

The entire Pod requests:

CPU = 300m

Memory = 384Mi

Maximum limits:

CPU = 600m

Memory = 768Mi

5. Deploy the Pod

Run:

kubectl apply -f resource-pod.yaml

Expected:

pod/resource-pod created

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 resource-pod.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: resource-pod  
  labels:  
    app: resource-demo  
spec:  
  containers:  
    - name: nginx1  
      image: nginx:latest  
      resources:  
        requests:  
          cpu: "100m"  
          memory: "128Mi"  
        limits:  
          cpu: "200m"  
          memory: "256Mi"  
    - name: nginx2  
      image: nginx:latest  
      command:  
        - /bin/sh  
        - -c  
        - |  
          sed -i 's/listen 80;/listen 8080;/' /etc/nginx/conf.d/default.conf  
          sed -i 's/listen \[:\]:80;/listen \[:\]:8080;/' /etc/nginx/conf.d/default.conf  
          nginx -g 'daemon off;'  
      resources:  
        requests:  
          cpu: "200m"  
          memory: "256Mi"  
        limits:  
          cpu: "400m"  
          memory: "512Mi"
```

6. Check Pod Status

Run:

```
kubectl get pod resource-pod
```

Expected:

NAME	READY	STATUS	RESTARTS	AGE
resource-pod	2/2	Running	0	...

2/2 means both containers are running.

7. Verify Resource Configuration

Now use:

```
kubectl describe pod resource-pod
```

Look for:

Containers:

nginx1:

Image: nginx:latest

Limits:

cpu: 200m

memory: 256Mi

Requests:

cpu: 100m

memory: 128Mi

nginx2:

Image: nginx:latest

Limits:

cpu: 400m

memory: 512Mi

Requests:

cpu: 200m

memory: 256Mi

This confirms that the resource configuration has been applied.

8. Check Only the Resource Information

You can also run:

```
kubectl describe pod resource-pod | grep -A8 -E "nginx1:|nginx2:"
```

However, for beginners, I recommend using:

```
kubectl describe pod resource-pod
```

and manually looking under the **Containers** section.

9. Verify Using YAML Output

Another useful command is:

```
kubectrl get pod resource-pod -o yaml
```

Search for:

resources:

You should see:

resources:

limits:

cpu: 200m

memory: 256Mi

requests:

cpu: 100m

memory: 128Mi

and for the second container:

resources:

limits:

cpu: 400m

memory: 512Mi

requests:

cpu: 200m

memory: 256Mi

10. Check Actual Resource Usage

If Metrics Server is installed in your cluster, you can run:

```
kubectrl top pod resource-pod
```

For individual containers:

```
kubectl top pod resource-pod --containers
```

Example:

POD	NAME	CPU(cores)	MEMORY(bytes)
resource-pod	nginx1	1m	8Mi
resource-pod	nginx2	1m	8Mi

requests and limits are configuration values. kubectl top shows actual usage. The Metrics Server must be installed for kubectl top to work.

11. Requests vs Limits — Important Interview Question

Requests

A **request** tells Kubernetes:

"This container needs at least this amount of CPU/memory for scheduling."

For example:

requests:

```
cpu: "100m"
```

```
memory: "128Mi"
```

Kubernetes uses requests when deciding **which worker node has enough available resources to schedule the Pod**.

Limits

A **limit** tells Kubernetes:

"This container should not use more than this configured amount."

For example:

limits:

```
cpu: "200m"
```

```
memory: "256Mi"
```

CPU can be throttled when it reaches its limit. A container that exceeds its memory limit can be terminated with an **OOMKilled** condition.

12. Requests vs Limits Diagram

Kubernetes Scheduler

|

v

Resource Request

CPU: 100m

RAM: 128Mi

|

v

Select Worker

|

v

+-----+

| Pod |

| |

| nginx1 |

| Request: |

| CPU 100m |

| RAM 128Mi |

| |

| Limit: |

| CPU 200m |

| RAM 256Mi |

+-----+